МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01. «Математика и компьютерные науки»

Отчет о выполнении лабораторной работы №1
По дисциплине: Теория алгоритмов
На тему: «Реализация двумерного клеточного автомата
с окрестностью фон Неймана»
Вариант 43

Выполнил студент
группы 5130201/40101 _____ Храмов В.Д.

Принял преподаватель _____ Востров А.В.

«____» _____ 2026 г.

Санкт-Петербург
2026

# Содержание

**Введение** **3**

**1 Математическое описание** **4**

1.1 Конечный автомат . . . . . 4

1.2 Клеточный автомат . . . . . 4

1.3 Окрестность фон Неймана . . . . . 6

1.4 Функция перехода и номер правила . . . . . 7

1.5 Граничные условия . . . . . 8

1.6 Синхронное обновление . . . . . 9

1.7 Сходимость и периодичность . . . . . 9

1.8 Классификация Вольфрама . . . . . 10

**2 Особенности реализации** **11**

2.1 Класс CellularAutomaton . . . . . 11

2.2 Класс Main . . . . . 16

**3 Анализ клеточного автомата** **20**

3.1 Свойства правила . . . . . 20

3.2 Простые паттерны . . . . . 21

3.3 Случайные начальные условия . . . . . 23

3.4 Периодичность . . . . . 25

3.5 Чувствительность к начальным условиям . . . . . 27

3.6 Классификация по Вольфраму . . . . . 27

**4 Результаты работы программы** **29**

**Заключение** **36**

**Список литературы** **38**

**Приложение. Исходный код программы** **39**

# Введение

В отчете представлено описание выполнения лабораторной работы №1 по дисциплине «Теория алгоритмов».

Необходимо реализовать двумерный клеточный автомат с окрестностью фон Неймана в соответствии с полученным номером №. Граничные условия — торроидальные. Пользователь определяет ширину поля и количество итераций. Необходимо учесть возможность ввода различных начальных условий (как вручную, так и случайным образом) по выбору пользователя. Реализация возможна в консоли или в графике. В итоге необходимо проанализировать свой клеточный автомат в отчете (его поведение, паттерны, «сходимость», периодичность, классификация по Вольфраму и т.д.). № = номер варианта · 11 · год рождения · месяц · день.

Для варианта 43 и даты рождения 02.11.2006 номер правила равен $№ = 43 \cdot 11 \cdot 2006 \cdot 11 \cdot 2 = 20\,874\,436$.

Программа реализована на языке Java в виде консольного приложения. В разделе 1 приведено математическое описание автомата и правила №, в разделе 2 — особенности реализации, в разделе 3 — анализ поведения автомата, в разделе 4 — примеры работы программы.

# 1 Математическое описание

## 1.1 Конечный автомат

Детерминированным конечным автоматом (автоматом Мура) называется шестерка

$$M = (Q, X, Y, \delta, \lambda, q_0),$$

где

- $Q$ — конечное множество состояний;
- $X$ — конечный входной алфавит;
- $Y$ — конечный выходной алфавит;
- $\delta : Q \times X \to Q$ — функция переходов;
- $\lambda : Q \to Y$ — функция выходов;
- $q_0 \in Q$ — начальное состояние.

Автомат работает в дискретном времени $t = 0, 1, 2, \ldots$: в каждый такт он получает входной символ $x(t) \in X$, переходит в новое состояние и выдает выходной символ, зависящий от текущего состояния:

$$q(t+1) = \delta\big(q(t),\ x(t)\big), \qquad y(t) = \lambda\big(q(t)\big), \qquad q(0) = q_0.$$

Функцию переходов конечного автомата можно задать таблицей размером $|Q| \times |X|$.

## 1.2 Клеточный автомат

Клеточный автомат — это совокупность одинаковых конечных автоматов (клеток), расположенных в узлах регулярной решетки и соединенных между собой так, что входной символ каждого автомата образуют выходные символы его соседей. Все автоматы работают синхронно, по общим тактам времени [1, 2].

Формально клеточный автомат задается тройкой

$$A = (L, N, M),$$

где

- $L$ — решетка клеток; в работе это квадратное поле $n \times n$, клетка задается парой индексов $(i, j)$, где $i$ — номер строки, $j$ — номер столбца, $i, j \in \{0, \ldots, n-1\}$; индексы берутся по модулю $n$ (тороидальные граничные условия, раздел 1.5);
- $N$ — окрестность, то есть шаблон, определяющий, с какими клетками соединен вход автомата клетки;
- $M = (Q, X, Y, \delta, \lambda, q_0)$ — конечный автомат, находящийся в каждой клетке решетки.

  В данной работе автомат клетки $M$ имеет следующий вид:
- $Q = \{0, 1\}$ — состояния клетки: 0 — «мертвая», 1 — «живая»;
- $Y = Q$, $\lambda(q) = q$ — клетка передает соседям свое состояние;
- $X = Q^4$ — входной символ клетки $(i, j)$ есть набор состояний ее соседей по окрестности фон Неймана (раздел 1.3):

$$x_{i,j}(t) = \big(q_{i-1,j}(t),\ q_{i,j+1}(t),\ q_{i,j-1}(t),\ q_{i+1,j}(t)\big) = (N, E, W, S),$$

  поэтому входной алфавит содержит $|X| = 2^4 = 16$ символов;
- $\delta : Q \times X \to Q$ — функция переходов, одинаковая для всех клеток; ее таблица содержит $|Q| \cdot |X| = 2 \cdot 16 = 32$ значения и задается номером правила (раздел 1.4). Функцию переходов удобно записывать как функцию пяти переменных — состояния клетки и состояний четырех соседей:

$$\delta\big(C, (N, E, W, S)\big) = f(C, N, E, W, S), \qquad f : Q^5 \to Q;$$

- $q_0$ — начальное состояние, свое для каждой клетки; совокупность начальных состояний всех клеток (начальная конфигурация) задается пользователем вручную или случайным образом.

Таким образом, все клетки являются копиями одного и того же конечного автомата и различаются только начальными состояниями. Работа клеточного автомата описывается системой

$$q_{i,j}(t+1) = \delta\big(q_{i,j}(t),\ x_{i,j}(t)\big), \qquad i, j \in \{0, \ldots, n-1\},$$

где все $n^2$ автоматов срабатывают одновременно.

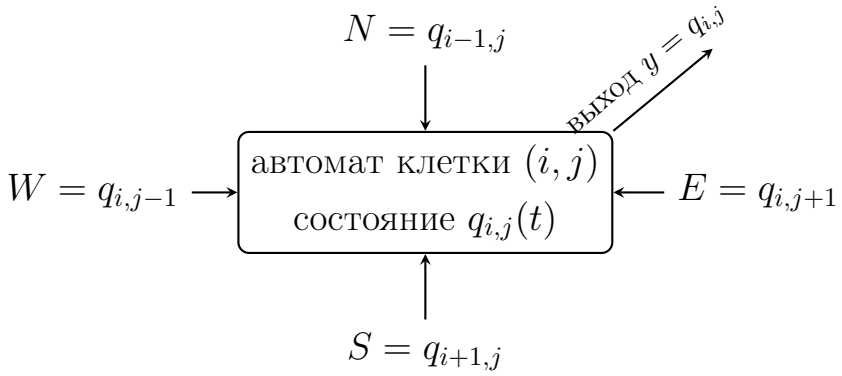



Рис. 1: Клетка как конечный автомат: вход — состояния четырех соседей, выход — собственное состояние

Совокупность состояний всех клеток в момент $t$ называется конфигурацией (поколением) $S^t$. Весь клеточный автомат на поле $n \times n$ также можно рассматривать как один конечный автомат без входа (автономный), состояниями которого являются конфигурации: их число конечно и равно $|Q|^{n^2} = 2^{n^2}$, а функция переходов $F$ переводит конфигурацию $S^t$ в $S^{t+1}$. Из конечности множества состояний следует, что последовательность конфигураций в конечном счете становится периодической (раздел 1.7).

## 1.3 Окрестность фон Неймана

В окрестность фон Неймана входят сама клетка $C$ и четыре клетки, имеющие с ней общую сторону: северная $N$, восточная $E$, южная $S$ и западная $W$ (рис. 2). Для клетки $(i, j)$:

$$C = (i, j), \quad N = (i-1, j), \quad E = (i, j+1), \quad S = (i+1, j), \quad W = (i, j-1).$$

Строки нумеруются сверху вниз, поэтому северный сосед находится в строке с меньшим номером.

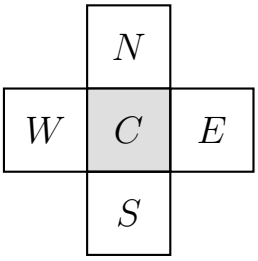



Рис. 2: Окрестность фон Неймана

## 1.4 Функция перехода и номер правила

Окрестность состоит из 5 клеток, каждая из которых имеет 2 состояния, поэтому существует $2^5 = 32$ различные конфигурации окрестности. Функция перехода $f$ сопоставляет каждой из них новое состояние центральной клетки, то есть полностью задается 32 битами. Всего существует $2^{32} \approx 4{,}3 \cdot 10^9$ таких правил.

Чтобы найти нужный бит, конфигурация окрестности переводится в индекс $0 \leq idx \leq 31$. В работе используется порядок $C\,N\,E\,W\,S$:

$$idx = 16C + 8N + 4E + 2W + S,$$

а новое состояние клетки равно биту номер $idx$ номера правила:

$$C_{i,j}^{t+1} = \left\lfloor \frac{№}{2^{idx}} \right\rfloor \bmod 2.$$

Номер правила в двоичной системе счисления (дополнен незначащими нулями до 32 разрядов):

$$20\,874\,436_{10} = 0000\,0001\,0011\,1110\,1000\,0100\,1100\,0100_2.$$

Таблица истинности функции $f$ приведена в таблице 1.

Таблица 1: Таблица истинности функции перехода (правило 20 874 436, порядок $C\,N\,E\,W\,S$)

| $idx$ | $C$ | $N$ | $E$ | $W$ | $S$ | $f$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 1 | 0 |
| 2 | 0 | 0 | 0 | 1 | 0 | 1 |
| 3 | 0 | 0 | 0 | 1 | 1 | 0 |
| 4 | 0 | 0 | 1 | 0 | 0 | 0 |
| 5 | 0 | 0 | 1 | 0 | 1 | 0 |
| 6 | 0 | 0 | 1 | 1 | 0 | 1 |
| 7 | 0 | 0 | 1 | 1 | 1 | 1 |
| 8 | 0 | 1 | 0 | 0 | 0 | 0 |
| 9 | 0 | 1 | 0 | 0 | 1 | 0 |
| 10 | 0 | 1 | 0 | 1 | 0 | 1 |
| 11 | 0 | 1 | 0 | 1 | 1 | 0 |
| 12 | 0 | 1 | 1 | 0 | 0 | 0 |
| 13 | 0 | 1 | 1 | 0 | 1 | 0 |
| 14 | 0 | 1 | 1 | 1 | 0 | 0 |
| 15 | 0 | 1 | 1 | 1 | 1 | 1 |

| $idx$ | $C$ | $N$ | $E$ | $W$ | $S$ | $f$ |
|---|---|---|---|---|---|---|
| 16 | 1 | 0 | 0 | 0 | 0 | 0 |
| 17 | 1 | 0 | 0 | 0 | 1 | 1 |
| 18 | 1 | 0 | 0 | 1 | 0 | 1 |
| 19 | 1 | 0 | 0 | 1 | 1 | 1 |
| 20 | 1 | 0 | 1 | 0 | 0 | 1 |
| 21 | 1 | 0 | 1 | 0 | 1 | 1 |
| 22 | 1 | 0 | 1 | 1 | 0 | 0 |
| 23 | 1 | 0 | 1 | 1 | 1 | 0 |
| 24 | 1 | 1 | 0 | 0 | 0 | 1 |
| 25 | 1 | 1 | 0 | 0 | 1 | 0 |
| 26 | 1 | 1 | 0 | 1 | 0 | 0 |
| 27 | 1 | 1 | 0 | 1 | 1 | 0 |
| 28 | 1 | 1 | 1 | 0 | 0 | 0 |
| 29 | 1 | 1 | 1 | 0 | 1 | 0 |
| 30 | 1 | 1 | 1 | 1 | 0 | 0 |
| 31 | 1 | 1 | 1 | 1 | 1 | 0 |

По таблице видно, что

- живая клетка выживает при 6 конфигурациях соседей из 16, мертвая рождается при 5 из 16;
- пустая окрестность остается пустой ($idx = 0 \to 0$), а клетка, у которой живы все 5 клеток окрестности, умирает ($idx = 31 \to 0$);
- правило не является тоталистическим, то есть зависит не от количества живых соседей, а от того, какие именно соседи живы. Например, мертвая клетка с одним живым западным соседом рождается ($idx = 2 \to 1$), а с одним живым восточным — нет ($idx = 4 \to 0$).

## 1.5 Граничные условия

В работе используются тороидальные граничные условия: противоположные края поля считаются соседними. Правый край поля склеивается с левым, а верхний — с нижним, и поле превращается в поверхность тора, у которой нет границ. Все клетки при этом оказываются равноправными: у каждой ровно 4 соседа.

Для поля $n \times n$ координата соседа, вышедшая за край, приводится по модулю $n$:

$$i' = i \bmod n, \qquad j' = j \bmod n,$$

где mod — математический остаток, лежащий в диапазоне $0, \ldots, n-1$. Например, для клеток на краях поля:

- верхняя строка ($i = 0$): северный сосед — клетка $(n-1, j)$;
- нижняя строка ($i = n-1$): южный сосед — клетка $(0, j)$;
- левый столбец ($j = 0$): западный сосед — клетка $(i, n-1)$;
- правый столбец ($j = n-1$): восточный сосед — клетка $(i, 0)$.

## 1.6 Синхронное обновление

Все клетки автомата обновляются одновременно: новое состояние каждой клетки вычисляется только по конфигурации $S^t$:

$$C_{i,j}^{t+1} = f\left(C_{i,j}^t,\ C_{i-1,j}^t,\ C_{i,j+1}^t,\ C_{i,j-1}^t,\ C_{i+1,j}^t\right).$$

Если записывать новые значения в то же поле, из которого читаются соседи, то часть клеток увидит соседей уже из поколения $t+1$, и результат будет зависеть от порядка обхода поля. Поэтому для вычисления шага используются два поля: из одного читается поколение $t$, в другое записывается поколение $t+1$.

## 1.7 Сходимость и периодичность

Поле $n \times n$ с двумя состояниями клетки имеет конечное число конфигураций — $2^{n^2}$. Автомат детерминирован: из одной конфигурации всегда получается одна и та же. Поэтому не более чем через $2^{n^2}$ шагов некоторая конфигурация обязательно повторится, и начиная с этого момента эволюция становится периодической:

$$S^0 \to S^1 \to \cdots \to S^{t_0} \to \cdots \to S^{t_0+p-1} \to S^{t_0+p} = S^{t_0}.$$

Здесь $t_0$ — предпериод (длина переходного процесса), $p$ — период. Если $p = 1$, конфигурация больше не меняется — это неподвижная точка. Если

$S^j = S^i$ для некоторых $i < j$ и все предыдущие конфигурации различны, то $t_0 = i$, $p = j - i$.

## 1.8 Классификация Вольфрама

С. Вольфрам предложил разделить клеточные автоматы на четыре класса по характеру их эволюции из случайных начальных конфигураций [3, 4]:

1. **Класс I** — эволюция приводит к однородному состоянию (все клетки мертвые или все живые).
2. **Класс II** — эволюция приводит к набору простых устойчивых или периодических структур, разделенных друг от друга.
3. **Класс III** — хаотическое (псевдослучайное) поведение. Конфигурации не упорядочиваются, малые изменения начальных условий распространяются по полю.
4. **Класс IV** — сложные локализованные структуры, которые движутся и взаимодействуют между собой; поведение на границе порядка и хаоса.

# 2 Особенности реализации

Программа написана на языке Java и состоит из двух классов:

- `CellularAutomaton` — логика автомата: хранение поля, правило перехода, шаг эволюции, начальные условия, обнаружение цикла;
- `Main` — консольный интерфейс: ввод параметров с проверкой, выбор начальных условий, вывод поля и итогов.

Полный исходный код приведен в Приложении.

## 2.1 Класс CellularAutomaton

Класс содержит следующие поля:

- `long rule` — номер правила;
- `int size` — ширина (и высота) квадратного поля;
- `boolean[][] cells` — текущее поколение, `cells[row][col]`, `true` — живая клетка;
- `boolean[][] next` — буфер для следующего поколения;
- `int generation` — номер текущего поколения;
- `Map<BitSet, Integer> history` — все встреченные конфигурации и номера поколений, в которых они впервые появились;
- `boolean cycleFound`, `int transientLength`, `int period` — признак найденного цикла, предпериод и период.

### Конструктор CellularAutomaton()

Конструктор вычисляет номер правила по формуле из задания и создает два пустых поля заданного размера. Приведение первого множителя к `long` гарантирует, что все произведение вычисляется в 64-битной арифметике без переполнения. Код представлен в листинге 1.

Входные данные: `int numberOption, constValue, year, month, day` — параметры формулы, `int size` — размер поля.

Выходные данные: объект автомата с пустым полем.

```
1 public CellularAutomaton(int numberOption, int constValue, int year, int
    month, int day, int size) {
2     this.rule = (long) numberOption * constValue * year * month * day;
3     this.size = size;
```

```
4       this.cells = new boolean[this.size][this.size];
5       this.next = new boolean[this.size][this.size];
6   }
```

Листинг 1: Конструктор класса CellularAutomaton

### Методы bit() и newState()

Метод `bit` возвращает $k$-й бит числа $x$: число сдвигается вправо на $k$ разрядов, после чего побитовым «И» с единицей выделяется младший бит. Метод `newState` применяет его к номеру правила и возвращает новое состояние клетки по индексу окрестности. Код представлен в листинге 2.

Входные данные: `long x` — число, `int k` — номер бита; для `newState` — индекс окрестности `idx` (0..31).

Выходные данные: 0 или 1.

```
1   int newState(int idx) {
2       return bit(rule, idx);
3   }
4
5   static int bit(long x, int k) {
6       long shifted = x >> k;
7       long lastBit = shifted & 1;
8       return (int) lastBit;
9   }
```

Листинг 2: Методы bit() и newState()

### Методы wrap() и getWrapped()

Метод `wrap` реализует тороидальные граничные условия: переводит любую координату в диапазон $0..size-1$. Метод `getWrapped` возвращает состояние клетки с учетом тора. Код представлен в листинге 3.

Входные данные: координата `coord` (для `getWrapped` — `row, col`), возможно, вышедшая за край поля.

Выходные данные: координата внутри поля; состояние клетки.

```
1   private int wrap(int coord) {
2       return (coord % size + size) % size;
3   }
4
5   private boolean getWrapped(int row, int col) {
6       return cells[wrap(row)][wrap(col)];
```

```
}
```

Листинг 3: Методы wrap() и getWrapped()

Метод neighborhoodIndex()

Метод собирает индекс окрестности фон Неймана клетки $(row, col)$ в порядке $C\,N\,E\,W\,S$. Состояние каждой из пяти клеток читается с учетом тора и переводится в 0 или 1 методом `toBit`, после чего биты складываются с весами 16, 8, 4, 2, 1. Код представлен в листинге 4.

Входные данные: `int row, col` — координаты клетки.

Выходные данные: индекс окрестности 0..31.

```
int neighborhoodIndex(int row, int col) {
    int n = toBit(getWrapped(row - 1, col));
    int c = toBit(getWrapped(row, col));
    int s = toBit(getWrapped(row + 1, col));
    int w = toBit(getWrapped(row, col - 1));
    int e = toBit(getWrapped(row, col + 1));
    return c * 16 + n * 8 + e * 4 + w * 2 + s;
}
```

Листинг 4: Метод neighborhoodIndex()

Метод step()

Метод выполняет один шаг эволюции. Для каждой клетки вычисляется индекс окрестности по текущему полю `cells`, новое состояние записывается в буфер `next`. После обработки всех клеток ссылки на массивы меняются местами: новое поколение становится текущим, а старый массив используется как буфер на следующем шаге. Копирования массивов при этом не происходит. Перед первым шагом в историю записывается начальная конфигурация, а после каждого шага выполняется проверка на повторение конфигурации. Сложность одного шага — $O(n^2)$. Код представлен в листинге 5.

Входные данные: текущее поле.

Выходные данные: поле следующего поколения, увеличенный номер поколения.

```
public void step() {
    if (history.isEmpty()) {
```

```
        history.put(snapshot(), generation);
    }

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            int idx = neighborhoodIndex(i, j);
            next[i][j] = newState(idx) == 1;
        }
    }

    boolean[][] tmp = cells;
    cells = next;
    next = tmp;

    generation++;
    checkCycle();
}
```

Листинг 5: Метод step()

Методы checkCycle() и snapshot()

Для обнаружения цикла все встреченные конфигурации хранятся в хеш-таблице `history` (конфигурация → номер поколения). Метод `snapshot` упаковывает поле в `BitSet`: клетка $(row, col)$ соответствует биту $row \cdot size + col$, т. е. на клетку тратится один бит. Методы `equals` и `hashCode` класса `BitSet` сравнивают содержимое, поэтому его можно использовать как ключ хеш-таблицы, в отличие от массива `boolean[][]`.

Метод `checkCycle` пытается добавить текущую конфигурацию в таблицу методом `putIfAbsent`. Если такая конфигурация уже встречалась в поколении $prev$, метод возвращает $prev$, и тогда предпериод равен $prev$, а период — $generation - prev$. После обнаружения цикла конфигурации больше не сохраняются. Сравнение конфигураций точное (без риска коллизий), а затраты памяти составляют $O(T \cdot n^2/8)$ байт, где $T$ — число выполненных шагов. Код представлен в листинге 6.

Входные данные: текущее поле, номер поколения.

Выходные данные: признак `cycleFound`, предпериод `transientLength` и период `period`.

```
private void checkCycle() {
    if (cycleFound) {
        return;
```

```
    }
    Integer prev = history.putIfAbsent(snapshot(), generation);
    if (prev != null) {
        cycleFound = true;
        transientLength = prev;
        period = generation - prev;
    }
}

private BitSet snapshot() {
    BitSet bits = new BitSet(size * size);
    for (int row = 0; row < size; row++) {
        for (int col = 0; col < size; col++) {
            if (cells[row][col]) {
                bits.set(row * size + col);
            }
        }
    }
    return bits;
}
```

Листинг 6: Методы checkCycle() и snapshot()

Метод randomize()

Метод заполняет поле случайным образом: каждая клетка становится живой с вероятностью `density`, независимо от остальных. Для этого используется равномерно распределенное число из $[0, 1)$, возвращаемое `Random.nextDouble()`: оно меньше `density` ровно с вероятностью `density`. Ожидаемое число живых клеток равно $density \cdot n^2$. Генератор псевдослучайных чисел инициализируется заданным значением `seed`, поэтому одинаковый `seed` всегда дает одинаковое поле, что позволяет воспроизводить эксперименты. Код представлен в листинге 7.

Входные данные: `double density` — плотность живых клеток (0..1), `long seed` — начальное значение генератора.

Выходные данные: случайно заполненное поле.

```
public void randomize(double density, long seed) {
    Random random = new Random(seed);
    for (int row = 0; row < size; row++) {
        for (int col = 0; col < size; col++) {
            cells[row][col] = random.nextDouble() < density;
        }
    }
```

```
}
```

Листинг 7: Метод randomize()

Также класс содержит методы `getCell` и `setCell` (чтение и запись клетки), `clear` (очистка поля), `population` (число живых клеток) и методы доступа `getSize`, `getGeneration`, `isCycleFound`, `getTransientLength`, `getPeriod`.

## 2.2 Класс Main

Класс `Main` реализует консольный интерфейс программы. Ввод и вывод выполняются с помощью класса `java.lang.IO`.

### Метод main()

Метод запрашивает у пользователя ширину поля (3..100), количество итераций (1..100 000), начальные условия, шаг вывода $k$ (печатается каждое $k$-е поколение) и необходимость остановки при обнаружении цикла. Затем выполняется заданное число итераций; печатаются начальное поле, каждое $k$-е поколение, последнее поколение и поколение, на котором обнаружен цикл. В конце выводится итог работы. Код представлен в листинге 8.

Минимальная ширина поля равна 3, так как при $n \leq 2$ на торе северный и южный (западный и восточный) соседи клетки совпадают, и окрестность фон Неймана вырождается.

```
printGeneration(automaton);
for (int i = 1; i <= iterations; i++) {
    boolean hadCycle = automaton.isCycleFound();
    automaton.step();
    boolean cycleJustFound = !hadCycle && automaton.isCycleFound();

    if (i % printEvery == 0 || i == iterations || cycleJustFound) {
        printGeneration(automaton);
    }
    if (cycleJustFound && stopOnCycle) {
        break;
    }
}

printSummary(automaton);
```

Листинг 8: Основной цикл метода main()

Методы readInt() и readLine()

Метод `readInt` запрашивает целое число до тех пор, пока не будет введено корректное значение из диапазона $[min, max]$. При пустом вводе возвращается значение по умолчанию, которое показывается в подсказке в квадратных скобках. Ввод читается целой строкой и преобразуется в число методом `Integer.parseInt`; некорректный ввод перехватывается через исключение `NumberFormatException`. Метод `readLine` читает строку и при окончании ввода (EOF) возвращает пустую строку. Код представлен в листинге 9.

Входные данные: `prompt` — подсказка, `min`, `max` — границы диапазона, `def` — значение по умолчанию.

Выходные данные: введенное число из $[min, max]$ или `def`.

```
static int readInt(String prompt, int min, int max, int def) {
    while (true) {
        String s = readLine(prompt + " [" + def + "]: ");
        if (s.isEmpty()) {
            return def;
        }
        try {
            int value = Integer.parseInt(s);
            if (value >= min && value <= max) {
                return value;
            }
        } catch (NumberFormatException ignored) {
        }
        println("  Введите целое число от " + min + " до " + max + ".");
    }
}

static String readLine(String prompt) {
    String s = readln(prompt);
    return s == null ? "" : s.trim();
}
```

Листинг 9: Методы readInt() и readLine()

Метод initField()

Метод выводит меню начальных условий и заполняет поле выбранным способом:

1. случайно — запрашиваются плотность живых клеток в процентах и `seed`. Если `seed` не введен, он генерируется из текущего времени и печатается, чтобы эксперимент можно было повторить (метод `readSeed`);
2. вручную — метод `readManual`;
3. одна живая клетка в центре поля;
4. пустое поле.

Метод readManual()

Метод реализует ручной ввод начальной конфигурации: пользователь вводит строки поля сверху вниз, символы `#` и `1` обозначают живую клетку, любые другие — мертвую. Строка короче ширины поля дополняется мертвыми клетками, лишние символы игнорируются, пустая строка завершает ввод (оставшиеся строки — мертвые). Код представлен в листинге 10.

Входные данные: автомат, поле которого заполняется.

Выходные данные: поле, заполненное по вводу пользователя.

```
static void readManual(CellularAutomaton automaton) {
    int size = automaton.getSize();
    println("Вводите строки поля сверху вниз (до " + size + " строк).");
    println("Живая клетка: # или 1, мёртвая: . или 0. Пустая строка - зако
       нчить ввод.");
    automaton.clear();
    for (int row = 0; row < size; row++) {
        String line = readln(String.format("%3d> ", row));
        if (line == null || line.isBlank()) {
            break;
        }
        for (int col = 0; col < Math.min(line.length(), size); col++) {
            char ch = line.charAt(col);
            automaton.setCell(row, col, ch == '#' || ch == '1');
        }
    }
}
```

Листинг 10: Метод readManual()

Методы вывода

Метод `printField` печатает поле построчно: `#` — живая клетка, `.` — мертвая. Метод `printGeneration` дополнительно печатает заголовок с номером поколения, числом живых клеток и, если цикл уже найден, предпериодом и периодом. Метод `printSummary` выводит итог: число выполненных итераций, число живых клеток, а также предпериод и период цикла либо сообщение о том, что цикл не обнаружен.

# 3 Анализ клеточного автомата

Все эксперименты, описанные в разделе, воспроизводятся в программе: случайные начальные условия задаются плотностью и значением `seed`, простые паттерны — ручным вводом.

## 3.1 Свойства правила

Из таблицы истинности (табл. 1) следуют свойства, которые определяют поведение автомата.

1. **Пустое поле неподвижно.** Конфигурация $idx = 0$ переходит в 0, поэтому из пустого поля ничего не возникает: $t_0 = 0$, $p = 1$.
2. **Полностью заполненное поле вымирает за один шаг.** Конфигурация $idx = 31$ переходит в 0, поэтому на следующем шаге все клетки мертвые, далее поле пустое: $t_0 = 1$, $p = 1$.
3. **Анизотропия.** Правило несимметрично относительно поворотов и отражений. Единственный сосед, присутствие которого вызывает рождение мертвой клетки, — западный ($idx = 2 \to 1$), тогда как одиночные северный, восточный или южный соседи рождения не вызывают ($idx = 8, 4, 1 \to 0$). В результате у динамики есть выделенное направление — на восток (вправо).
4. **Одномерное инвариантное подмножество.** Если живые клетки находятся только в одной строке, то клетки соседних строк видят лишь одного живого соседа по вертикали ($idx = 1$ или $idx = 8$) и не рождаются. Значит, конфигурация, занимающая одну строку, остается в этой строке. Внутри строки ($N = S = 0$) правило сводится к одномерному элементарному клеточному автомату с окрестностью $(W, C, E)$:

   $$C^{t+1} = W^t \oplus (C^t \wedge E^t),$$

   что соответствует элементарному правилу 120 по нумерации Вольфрама (зеркальное отражение правила 106).

## 3.2 Простые паттерны

### Одиночная клетка

Одиночная живая клетка умирает ($idx = 16 \to 0$), а клетка справа от нее рождается, так как видит живого западного соседа ($idx = 2 \to 1$). Остальные соседи живой клетки не рождаются. В результате клетка за каждый шаг сдвигается на одну позицию вправо (рис. 3), т. е. является простейшим «глайдером», движущимся с максимально возможной для окрестности фон Неймана скоростью. Дойдя до правого края, клетка появляется у левого (тор), поэтому на поле $n \times n$ она возвращается в исходное положение через $n$ шагов: $t_0 = 0$, $p = n$. На рис. 3 показано поле $7 \times 7$, на котором период равен 7.

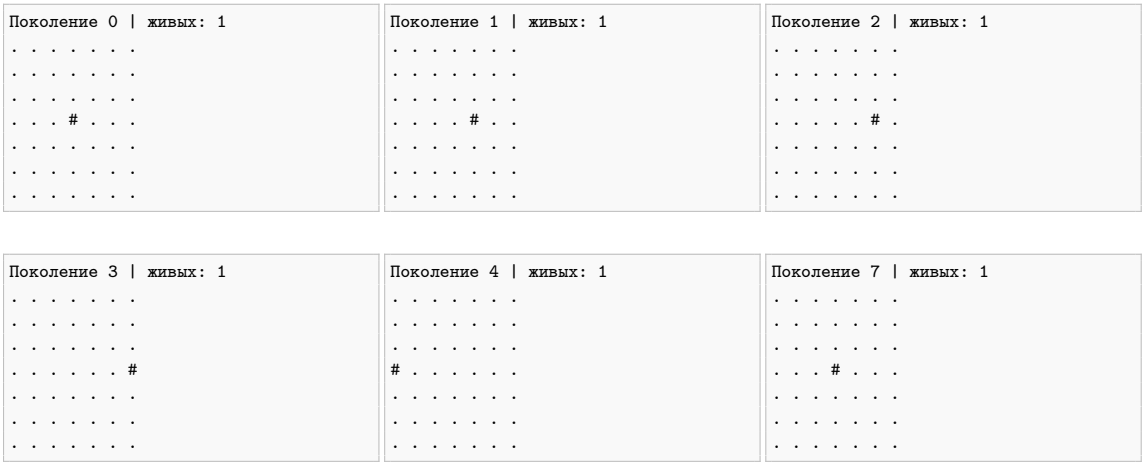



Рис. 3: Движение одиночной клетки на поле $7 \times 7$

### Горизонтальные паттерны

Паттерн из живых клеток одной строки, согласно свойству 4, развивается только внутри этой строки по одномерному правилу 120. На рис. 4 показана эволюция пары соседних клеток: в строке возникает нерегулярная последовательность живых клеток, которая смещается вправо. На поле $20 \times 20$ такая эволюция выходит на цикл с периодом 5240 после предпериода 785 шагов (табл. 2).

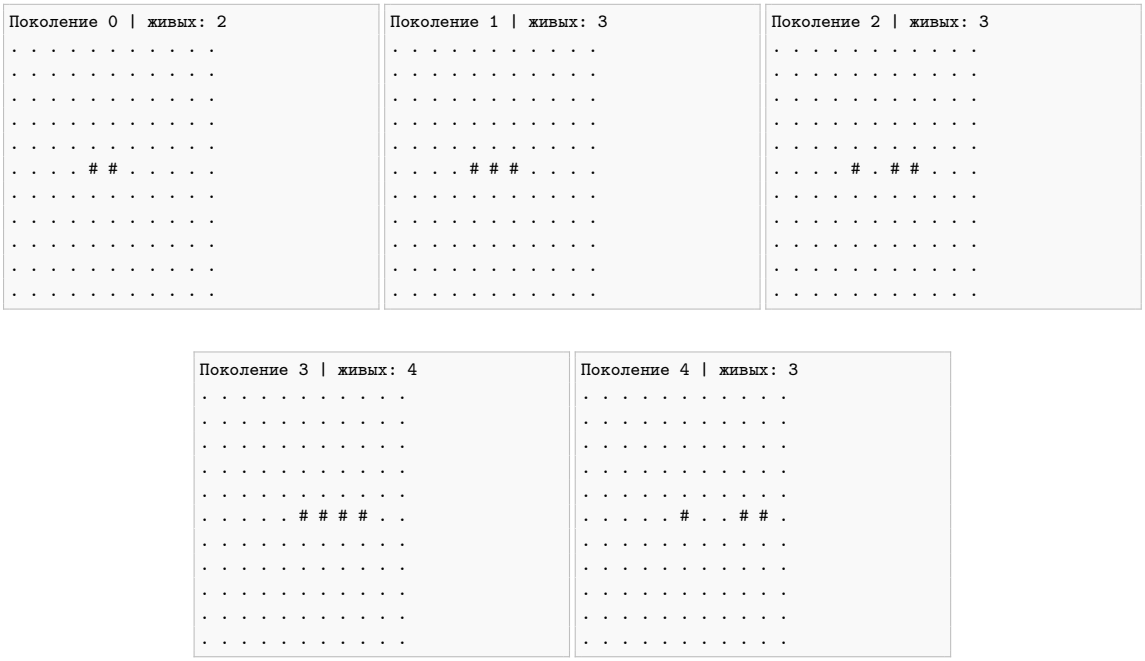



Рис. 4: Эволюция горизонтальной пары клеток на поле $11 \times 11$

Двумерные паттерны

Если живые клетки находятся в нескольких строках, то возникают конфигурации с живым западным соседом в соседних строках, и паттерн начинает расти по вертикали. На рис. 5 и 6 показаны первые шаги эволюции вертикальной пары клеток и «креста». В обоих случаях паттерн разрастается и одновременно смещается вправо.

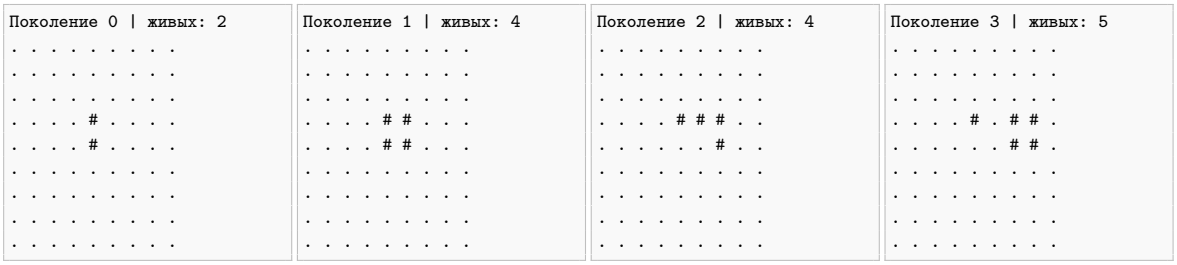



Рис. 5: Эволюция вертикальной пары клеток на поле $9 \times 9$

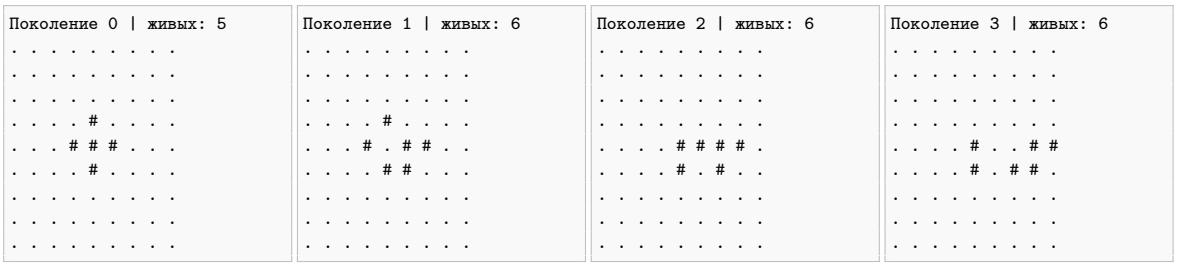



Рис. 6: Эволюция паттерна «крест» на поле $9 \times 9$

Результаты запуска автомата на простых паттернах, помещенных в центр поля $20 \times 20$, приведены в таблице 2.

Таблица 2: Простые паттерны на поле $20 \times 20$

| Паттерн | Предпериод $t_0$ | Период $p$ | Живых в цикле |
|---|---|---|---|
| одна клетка | 0 | 20 | 1 |
| диагональ из 3 клеток | 1 | 20 | 1 |
| крест | 11 | 40 | 9 |
| две клетки по горизонтали | 785 | 5240 | 10 |
| горизонтальная линия из 3 клеток | 784 | 5240 | 10 |
| две клетки по вертикали | 428 | 21240 | 13 |
| квадрат $2 \times 2$ | 427 | 21240 | 13 |
| квадрат $3 \times 3$ | 1036 | 5240 | 21 |
| буква «L» | 786 | 5240 | 20 |
| вертикальная линия из 3 клеток | не найден за $10^5$ шагов | | 25 |

Видно, что даже паттерны из 2–5 клеток в большинстве случаев проходят длинный переходный процесс (сотни шагов) и выходят на циклы длиной в тысячи шагов. Исключения — паттерны, которые за несколько шагов сводятся к одиночной движущейся клетке (диагональ) или к небольшой движущейся структуре (крест).

## 3.3 Случайные начальные условия

На рис. 7 показана эволюция случайной начальной конфигурации на поле $30 \times 30$ с плотностью живых клеток 30 % (`seed` $= 1$). Уже через несколько шагов поле перестает напоминать начальную конфигурацию, и далее картина остается неупорядоченной: устойчивых или периодических локальных структур не возникает, характерны короткие горизонтальные цепочки живых клеток, движущиеся вправо.

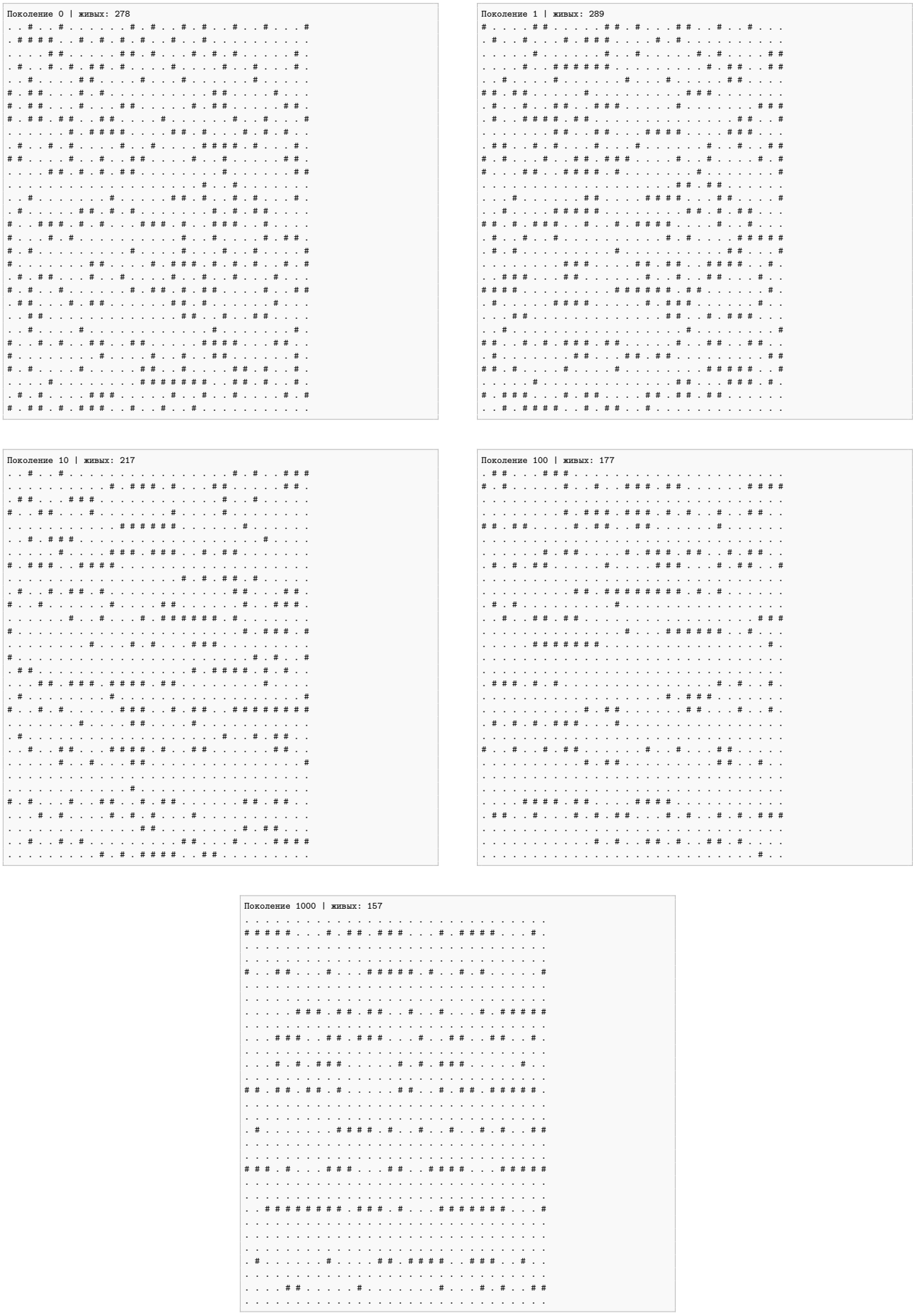



Рис. 7: Эволюция случайной конфигурации на поле $30 \times 30$ (плотность 30 %, seed = 1)

Для оценки «сходимости» была исследована доля живых клеток в зависимости от номера поколения. Эксперимент проводился на поле $50 \times 50$ для начальных плотностей 10, 30, 50, 70 и 90 %; для каждой плотности результат усреднен по 20 запускам (`seed` $= 1..20$). Результаты представлены на рис. 8.

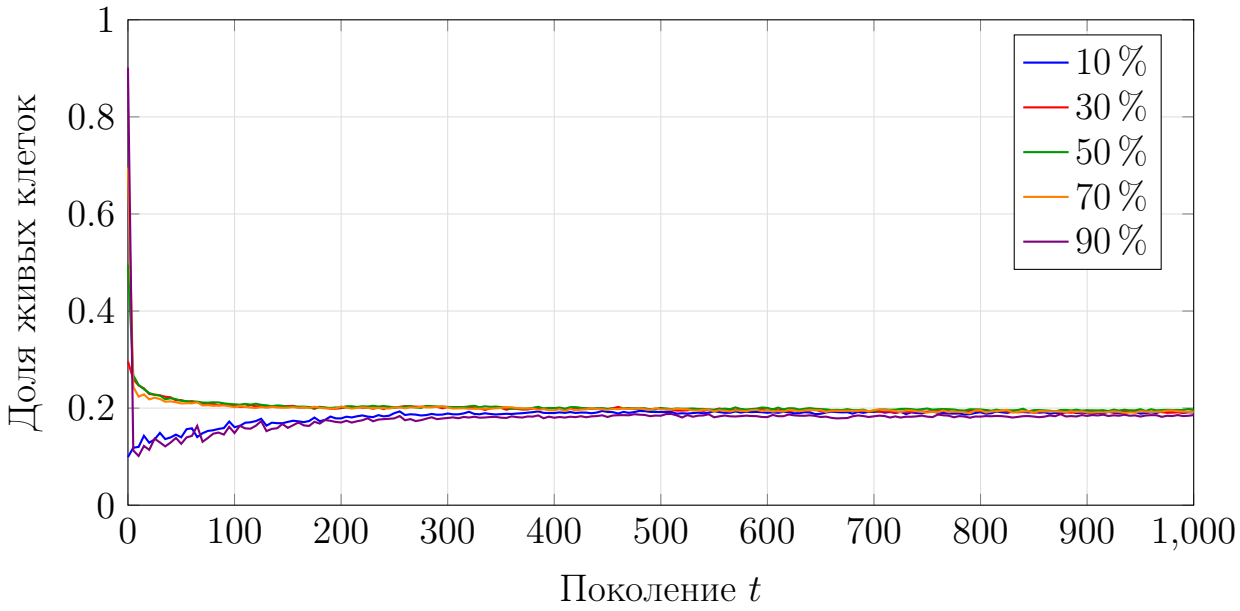



Рис. 8: Доля живых клеток на поле $50 \times 50$ (среднее по 20 запускам)

Независимо от начальной плотности доля живых клеток стремится к одному и тому же значению $\approx 0{,}19$–$0{,}20$. При плотностях 30–70 % это значение достигается за 100–200 шагов. При плотности 90 % большинство клеток окружено живыми соседями и гибнет на первом же шаге ($idx = 31 \to 0$), поэтому доля резко падает до $\approx 0{,}1$ и затем медленно растет; при плотности 10 % доля медленно растет снизу. Таким образом, автомат «забывает» начальную плотность и выходит на статистически стационарный режим, однако сама конфигурация при этом продолжает непрерывно меняться.

## 3.4 Периодичность

В таблице 3 приведены предпериоды и периоды, найденные программой для полей $n \times n$ при $n = 3..16$, плотности 50 % и `seed` $= 1..10$ (ограничение — $10^5$ шагов, как в программе).

Таблица 3: Предпериоды и периоды для случайных конфигураций (плотность 50 %, 10 запусков)

| $n$ | $2^{n^2}$ | $t_0$ (мин–макс) | Найденные периоды $p$ | Не найден |
|---|---|---|---|---|
| 3 | $5{,}1 \cdot 10^2$ | 0–5 | 1, 3 | 0 |
| 4 | $6{,}6 \cdot 10^4$ | 1–14 | 2, 4, 18 | 0 |
| 5 | $3{,}4 \cdot 10^7$ | 1–12 | 5, 15, 45 | 0 |
| 6 | $6{,}9 \cdot 10^{10}$ | 8–24 | 1, 3, 6 | 0 |
| 7 | $5{,}6 \cdot 10^{14}$ | 2–46 | 7, 28, 49, 196 | 0 |
| 8 | $1{,}8 \cdot 10^{19}$ | 10–57 | 30, 36, 120, 180, 360 | 0 |
| 9 | $2{,}4 \cdot 10^{24}$ | 30–154 | 9, 36, 54, 108 | 0 |
| 10 | $1{,}3 \cdot 10^{30}$ | 30–118 | 205, 410, 1640 | 0 |
| 11 | $2{,}7 \cdot 10^{36}$ | 47–139 | 176, 1232 | 0 |
| 12 | $2{,}2 \cdot 10^{43}$ | 50–311 | 1, 12, 24, 168 | 0 |
| 13 | $7{,}5 \cdot 10^{50}$ | 125–190 | 416, 4576, 12896, 22880 | 5 |
| 14 | $1{,}0 \cdot 10^{59}$ | 179–427 | 15680, 28224 | 2 |
| 15 | $5{,}4 \cdot 10^{67}$ | 134–289 | 450, 32850, 59850 | 7 |
| 16 | $1{,}2 \cdot 10^{77}$ | 189–745 | 2688, 13440 | 0 |

На полях $n = 17, 18$ циклы еще находятся (например, $t_0 = 746$, $p = 2533$ при $n = 17$; $t_0 = 554$, $p = 23\,976$ при $n = 18$), но уже при $n = 20$ и $n = 25$ ни в одном из запусков цикл за $10^5$ шагов не обнаружен; при $n = 30$ (плотность 30 %) цикл не найден за $10^5$ шагов.

Из таблицы видно, что

- предпериод невелик по сравнению с периодом: автомат быстро выходит на аттрактор, но аттрактор оказывается очень длинным циклом;
- период в целом быстро растет с размером поля (от единиц при $n \leq 6$ до десятков тысяч при $n \geq 13$), хотя рост неравномерен и зависит от арифметических свойств $n$;
- периоды часто кратны $n$, что связано со сдвигом структур вправо: конфигурация возвращается в исходное положение после целого числа оборотов вокруг тора;
- найденные периоды несравнимо меньше числа $2^{n^2}$ возможных конфигураций, однако при $n \geq 20$ они уже недоступны для прямого перебора.

## 3.5 Чувствительность к начальным условиям

Для проверки чувствительности к начальным условиям на поле $100 \times 100$ запускались два экземпляра автомата из одинаковой случайной конфигурации (плотность 50 %), отличающихся состоянием одной центральной клетки. На каждом шаге вычислялось расстояние Хэмминга между конфигурациями — число клеток, в которых они различаются. Результат, усредненный по 10 запускам, представлен на рис. 9.

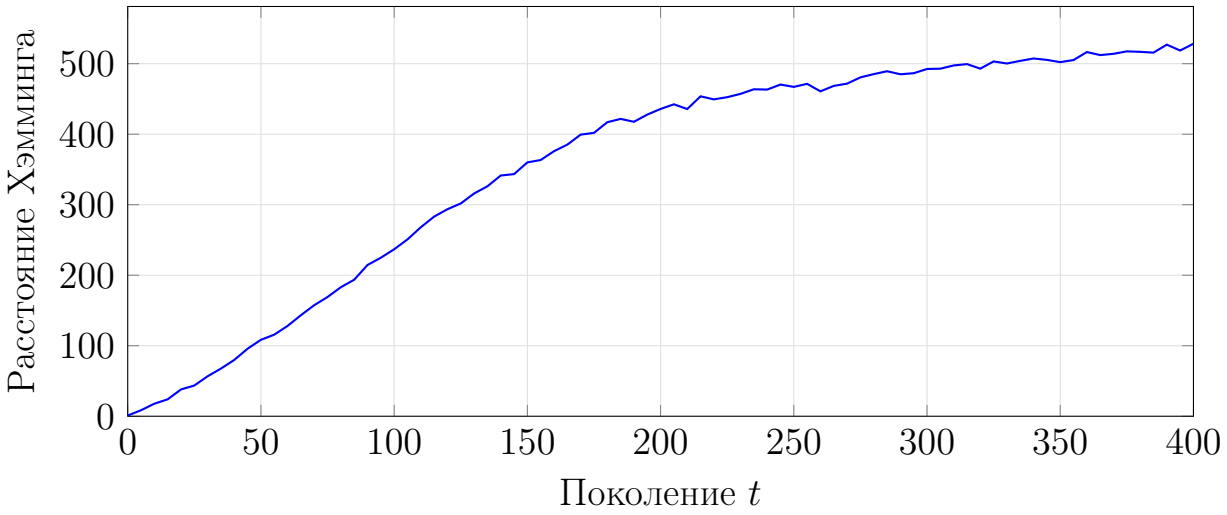



Рис. 9: Распространение возмущения одной клетки на поле $100 \times 100$ (среднее по 10 запускам)

Возмущение одной клетки не затухает, а разрастается: за первые 120 шагов число различающихся клеток растет примерно линейно и достигает $\approx 300$, после чего рост замедляется. Область возмущения смещается вправо со скоростью одна клетка за шаг и расширяется преимущественно по горизонтали, а по вертикали — медленно, что является следствием анизотропии правила. В отдельных запусках возмущение исчезает, не успев распространиться.

## 3.6 Классификация по Вольфраму

По результатам анализа автомат с правилом 20 874 436 (порядок $C\,N\,E\,W\,S$) относится к **классу III** (хаотическое поведение):

- эволюция случайных конфигураций не приводит ни к однородному состоянию (класс I), ни к набору простых устойчивых или периоди-

ческих структур (класс II): поле остается неупорядоченным, а доля живых клеток колеблется около $\approx 0{,}2$;

- малые изменения начальных условий распространяются по полю (рис. 9);
- длины циклов на торе быстро растут с размером поля и уже при $n \geq 20$ не наблюдаются за $10^5$ шагов;
- даже простые паттерны из нескольких клеток порождают длительную нерегулярную эволюцию.

У автомата есть и черты других классов: на малых полях ($n \leq 6$) встречаются неподвижные точки и короткие циклы, а одиночная клетка является движущейся локализованной структурой («глайдером»). Однако на случайных начальных конфигурациях такие структуры не сохраняются и растворяются в хаотическом фоне, поэтому оснований для отнесения автомата к классу IV нет.

# 4 Результаты работы программы

Ниже приведены листинги работы программы (вывод в консоль вместе с введенными пользователем значениями). Программа запускается командой `./gradlew run` из каталога проекта.

После запуска программа выводит заголовок, запрашивает ширину поля и количество итераций, а затем предлагает выбрать способ задания начальных условий (рис. 10). В квадратных скобках указаны значения по умолчанию, которые принимаются при пустом вводе.

```
Двумерный клеточный автомат

Ширина поля (3...100) [20]: 20
Количество итераций (1...100000) [10]: 1

Начальные условия:
  1 - случайно
  2 - вручную
  3 - одна клетка в центре
  4 - пустое поле
Выбор [1]: 1
```

Рис. 10: Запуск программы и ввод параметров

Все значения вводятся с проверкой: если введено не целое число (`abc`, `2.5`) или число вне допустимого диапазона (`-5`, `1000`), программа выводит сообщение с допустимым диапазоном и повторяет запрос. Для `seed` допускается любое целое число или пустой ввод. На рис. 11 показан пример запуска, в котором для каждого параметра сначала было введено некорректное значение, а затем корректное. После ввода всех параметров программа продолжает работу в обычном режиме.

```
Двумерный клеточный автомат

Ширина поля (3...100) [20]: abc
  Введите целое число от 3 до 100.
Ширина поля (3...100) [20]: -5
  Введите целое число от 3 до 100.
Ширина поля (3...100) [20]: 1000
  Введите целое число от 3 до 100.
Ширина поля (3...100) [20]: 5
Количество итераций (1...100000) [10]: 2.5
  Введите целое число от 1 до 100000.
Количество итераций (1...100000) [10]: 0
  Введите целое число от 1 до 100000.
Количество итераций (1...100000) [10]: 3

Начальные условия:
  1 - случайно
  2 - вручную
  3 - одна клетка в центре
  4 - пустое поле
Выбор [1]: 7
  Введите целое число от 1 до 4.
Выбор [1]: 1
Плотность живых клеток, % (0..100) [50]: 150
  Введите целое число от 0 до 100.
Плотность живых клеток, % (0..100) [50]: 30
Seed (пусто = случайный): abc
  Введите целое число или оставьте пустым.
Seed (пусто = случайный): 7

Печатать каждое k-е поколение, k [1]: x
  Введите целое число от 1 до 100000.
Печатать каждое k-е поколение, k [1]: 3
Остановиться, когда найден цикл? (1 - да, 0 - нет) [1]: 2
  Введите целое число от 0 до 1.
Остановиться, когда найден цикл? (1 - да, 0 - нет) [1]: 1
```

Рис. 11: Обработка некорректного ввода (фрагмент вывода программы)

При выборе случайных начальных условий пользователь задает плотность живых клеток в процентах и `seed` генератора псевдослучайных чисел; при пустом вводе `seed` генерируется автоматически и выводится на экран, чтобы запуск можно было повторить. Затем задаются частота вывода поколений и необходимость остановки при обнаружении цикла, после чего выводится начальное поколение. На рис. 12 показано продолжение запуска с рис. 10: поле $20 \times 20$, плотность $30\,\%$, seed $= 1$; в начальном поколении 118 живых клеток (ожидаемое значение $0{,}3 \cdot 400 = 120$).

```
Плотность живых клеток, % (0..100) [50]: 30
Seed (пусто = случайный): 1

Печатать каждое k-е поколение, k [1]: 1
Остановиться, когда найден цикл? (1 - да, 0 - нет) [1]: 1

Поколение 0 | живых: 118
. . # . . # . . . . . . # . # . . # . #
. . # . . # . . . # . # # # # . . # . #
. # . # . . # . . # . . . . . . . . . .
. . . . # # . . . . . # # . # . . . # .
# . # . . . . . # . . # . . # . # . # #
. # . . . . # . . . . # . . # . . . # .
. . # . . . . # # . . . . # . . . # . .
. . . . # . . . . . # . # # . . . # . #
. . . . . . . . . . # # . . . . # . . .
# . # # . . . # . . . # # . . . . . # .
# # . . . . . # # . # . # # . # # . . #
# . . . . # . . . . . . # . . # . . . #
. . . . . . # . # # # # . . . . # # . #
. . . # . # . # . . . # . . # . # . . .
. # . . # . . . . # # # # . # . . . # .
# # . . . . # . . # . . # # . . . . # .
. # . . . . . # # . . . . . # # . # . #
. # # . . . . . . . . # . . . . . . # #
. . . . . . . . . . . . . . . . . . . #
. . # . . . . . . . . . # . . . . . . .
```

Рис. 12: Случайное заполнение поля (продолжение рис. 10)

При ручном вводе пользователь вводит строки поля сверху вниз: символы `#` и `1` обозначают живую клетку, остальные — мертвую; пустая строка завершает ввод, недостающие строки и столбцы считаются мертвыми (рис. 13). В примере на поле $9 \times 9$ введен паттерн «крест».

```
Выбор [1]: 2
Вводите строки поля сверху вниз (до 9 строк).
Живая клетка: # или 1, мёртвая: . или 0. Пустая строка - закончить ввод.
  0> .........
  1> .........
  2> .........
  3> ....#....
  4> ...###...
  5> ....#....
  6>

Печатать каждое k-е поколение, k [1]: 1
Остановиться, когда найден цикл? (1 - да, 0 - нет) [1]: 0

Поколение 0 | живых: 5
. . . . . . . . .
. . . . . . . . .
. . . . . . . . .
. . . . # . . . .
. . . # # # . . .
. . . . # . . . .
. . . . . . . . .
. . . . . . . . .
. . . . . . . . .
```

Рис. 13: Ручной ввод начальной конфигурации

На рис. 14 показано продолжение этого запуска: вывод поколений 1–3 и итог работы. Видно, что паттерн смещается вправо, как описано в разделе 3 (рис. 6). Так как остановка при обнаружении цикла не выбрана и за 3 итерации цикл не найден, итог содержит соответствующее сообщение.

```
Поколение 1 | живых: 6
. . . . . .
. . . . . .
. . . . . .
. . . # . . .
. . # . # # .
. . . # # . .
. . . . . .
. . . . . .
. . . . . .

Поколение 2 | живых: 6
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . # # # .
. . . # . # .
. . . . . .
. . . . . .
. . . . . .

Поколение 3 | живых: 6
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . # . . # #
. . . # . # # .
. . . . . .
. . . . . .
. . . . . .

Итог
Выполнено итераций: 3
Живых клеток: 6 из 81
Цикл за 3 итераций не обнаружен.
```

Рис. 14: Вывод поколений и итог работы (продолжение рис. 13)

При обнаружении повторения конфигурации программа выводит поколение, на котором найден цикл, с предпериодом и периодом в заголовке, а затем итог работы (рис. 15). В примере одиночная клетка в центре поля $7 \times 7$ за 7 шагов обходит тор и возвращается в исходное положение: предпериод равен 0, период — 7. Чтобы не выводить промежуточные поколения, задано $k = 100$; поколение, на котором найден цикл, выводится всегда.

```
Двумерный клеточный автомат

Ширина поля (3...100) [20]: 7
Количество итераций (1...100000) [10]: 100

Начальные условия:
  1 - случайно
  2 - вручную
  3 - одна клетка в центре
  4 - пустое поле
Выбор [1]: 3

Печатать каждое k-е поколение, k [1]: 100
Остановиться, когда найден цикл? (1 - да, 0 - нет) [1]: 1

Поколение 0 | живых: 1
. . . . . .
. . . . . .
. . . . . .
. . . # . . .
. . . . . .
. . . . . .
. . . . . .

Поколение 7 | живых: 1 | цикл: предпериод = 0, период = 7
. . . . . .
. . . . . .
. . . . . .
. . . # . . .
. . . . . .
. . . . . .
. . . . . .

Итог
Выполнено итераций: 7
Живых клеток: 1 из 49
Цикл найден на поколении 7
Предпериод: 0
Период: 7
```

Рис. 15: Обнаружение цикла и итог работы

Если цикл за заданное число итераций не найден, программа сообщает об этом (рис. 16). Запуск выполнен на поле $20 \times 20$ со случайными начальными условиями (плотность 30 %, seed $= 1$), 100 000 итераций и $k = 100\,000$, поэтому выведены только начальное (на рисунке не показано) и последнее поколения.

```
Поколение 100000 | живых: 68
. . . . . .
. # . # # . . . # . . . # . . . . . .
. . . . . .
# . # . . . . . # # # # . . . . # # #
. . . . . .
. . . . . .
. . . . . .
. . # . . . # # . . . . # . # # # # # #
. . . . . .
. . . . . .
. . . . . .
# . # . . # # # # # . # . # # # # . # #
. . . . . .
# # . # . # . # # . . # # . . # # # # .
. . . . . .
# . # # . # . . # # . # . . . # # # . #
. . . . . .
. . . . . .
. . . . . .
. # . . # . # . . # # . . # . . . # . .

Итог
Выполнено итераций: 100000
Живых клеток: 68 из 400
Цикл за 100000 итераций не обнаружен.
```

Рис. 16: Итог работы, цикл не обнаружен (фрагмент: последнее поколение и итог)

# Заключение

В ходе выполнения лабораторной работы был реализован двумерный клеточный автомат с окрестностью фон Неймана, тороидальными граничными условиями и правилом перехода 20 874 436. Программа позволяет задавать ширину поля и количество итераций, выбирать случайные (с заданной плотностью и `seed`) или ручные начальные условия, а также автоматически обнаруживает цикл и вычисляет его предпериод и период.

Анализ показал, что правило анизотропно: рождение клетки вызывает только западный сосед, поэтому все структуры смещаются вправо, а одиночная клетка является глайдером с периодом $n$ на торе. Конфигурации, занимающие одну строку, эволюционируют по одномерному элементарному правилу 120. Случайные конфигурации независимо от начальной плотности выходят на режим с долей живых клеток $\approx 0{,}2$, но остаются неупорядоченными; длины циклов быстро растут с размером поля, а возмущение одной клетки распространяется по полю. По классификации Вольфрама автомат отнесен к классу III (хаотическое поведение).

## Достоинства реализации

- Логика автомата (класс `CellularAutomaton`) отделена от ввода-вывода (класс `Main`), поэтому автомат можно использовать и тестировать без консоли.
- Синхронное обновление реализовано обменом ссылок на два массива без копирования, сложность шага — $O(n^2)$.
- Обнаружение цикла точное: конфигурации сравниваются целиком (`BitSet`), без риска ложных совпадений, характерного для хранения только хеш-значений.
- Ввод устойчив к ошибкам: некорректные значения и выход за диапазон приводят к повторному запросу, а при окончании ввода используются значения по умолчанию.
- Случайные начальные условия воспроизводимы благодаря `seed`, который печатается, даже если был сгенерирован автоматически.

## Недостатки реализации

- Хранение всех конфигураций требует памяти, пропорциональной числу шагов: при поле $100 \times 100$ и $10^5$ шагах это порядка сотен мегабайт.
- Номер правила и порядок клеток в индексе заданы в коде; изменить их без перекомпиляции нельзя.
- Поле выводится символами в консоль, поэтому большие поля ($n > 60$) неудобно просматривать, а графики (например, доли живых клеток) программа не строит.
- Поддерживается только квадратное поле.

## Масштабирование

- Поиск цикла алгоритмом Брента или Флойда, требующим $O(1)$ дополнительной памяти, что позволит исследовать периоды на больших полях.
- Ввод номера правила и порядка клеток в индексе пользователем, поддержка прямоугольных полей и окрестности Мура.
- Графический интерфейс с анимацией и построением графиков статистики (доли живых клеток, расстояния Хэмминга).
- Параллельное вычисление шага для больших полей (строки поля обрабатываются независимо).

Работа выполнена в среде IntelliJ IDEA на языке Java (JDK 26). Код программы представлен в Приложении.

# Список литературы

[1] Секция «Телематика». https://tema.spbstu.ru/algorithm/. (Дата обращения: 24.10.2026).

[2] Тоффоли Т., Марголус Н. Машины клеточных автоматов. — М.: Мир, 1991. — 280 с.

[3] Wolfram S. Universality and complexity in cellular automata // Physica D: Nonlinear Phenomena. — 1984. — Vol. 10, no. 1–2. — P. 1–35.

[4] Wolfram S. A New Kind of Science. — Champaign: Wolfram Media, 2002. — 1197 p.

# Приложение. Исходный код программы

## Класс CellularAutomaton

```
package ru.spbstu.khramov;

import java.util.BitSet;
import java.util.HashMap;
import java.util.Map;
import java.util.Random;

public class CellularAutomaton {
    private final long rule;
    private boolean[][] cells;
    private final int size;
    private boolean[][] next;

    private int generation;

    private final Map<BitSet, Integer> history = new HashMap<>();
    private boolean cycleFound;
    private int transientLength;
    private int period;

    public CellularAutomaton(int numberOption, int constValue, int year,
        int month, int day, int size) {
        this.rule = (long) numberOption * constValue * year * month * day;
        this.size = size;
        this.cells = new boolean[this.size][this.size];
        this.next = new boolean[this.size][this.size];
    }

    int newState(int idx) {
        return bit(rule, idx);
    }

    static int bit(long x, int k) {
        long shifted = x >> k;
        long lastBit = shifted & 1;
        return (int) lastBit;
    }

    public boolean getCell(int row, int col) {
        return cells[row][col];
    }

    public void setCell(int row, int col, boolean value) {
```

```
43          cells[row][col] = value;
44      }
45
46      private int wrap(int coord) {
47          return (coord % size + size) % size;
48      }
49
50      private boolean getWrapped(int row, int col) {
51          return cells[wrap(row)][wrap(col)];
52      }
53
54      private static int toBit(boolean value) {
55          return value ? 1 : 0;
56      }
57
58      int neighborhoodIndex(int row, int col) {
59          int n = toBit(getWrapped(row - 1, col));
60          int c = toBit(getWrapped(row, col));
61          int s = toBit(getWrapped(row + 1, col));
62          int w = toBit(getWrapped(row, col - 1));
63          int e = toBit(getWrapped(row, col + 1));
64          return c * 16 + n * 8 + e * 4 + w * 2 + s;
65      }
66
67      public int getGeneration() {
68          return generation;
69      }
70
71      public int getSize() {
72          return size;
73      }
74
75      public void step() {
76          if (history.isEmpty()) {
77              history.put(snapshot(), generation);
78          }
79
80          for (int i = 0; i < size; i++) {
81              for (int j = 0; j < size; j++) {
82                  int idx = neighborhoodIndex(i, j);
83                  next[i][j] = newState(idx) == 1;
84              }
85          }
86
87          boolean[][] tmp = cells;
88          cells = next;
89          next = tmp;
90
```

```
91          generation++;
92          checkCycle();
93      }
94
95      private void checkCycle() {
96          if (cycleFound) {
97              return;
98          }
99          Integer prev = history.putIfAbsent(snapshot(), generation);
100         if (prev != null) {
101             cycleFound = true;
102             transientLength = prev;
103             period = generation - prev;
104         }
105     }
106
107     private BitSet snapshot() {
108         BitSet bits = new BitSet(size * size);
109         for (int row = 0; row < size; row++) {
110             for (int col = 0; col < size; col++) {
111                 if (cells[row][col]) {
112                     bits.set(row * size + col);
113                 }
114             }
115         }
116         return bits;
117     }
118
119     public boolean isCycleFound() {
120         return cycleFound;
121     }
122
123     public int getTransientLength() {
124         return transientLength;
125     }
126
127     public int getPeriod() {
128         return period;
129     }
130
131     public int population() {
132         int count = 0;
133         for (int row = 0; row < size; row++) {
134             for (int col = 0; col < size; col++) {
135                 if (cells[row][col]) {
136                     count++;
137                 }
138             }
```

```
139         }
140         return count;
141     }
142 
143     public void clear() {
144         for (int row = 0; row < size; row++) {
145             for (int col = 0; col < size; col++) {
146                 cells[row][col] = false;
147             }
148         }
149     }
150 
151     public void randomize(double density, long seed) {
152         Random random = new Random(seed);
153         for (int row = 0; row < size; row++) {
154             for (int col = 0; col < size; col++) {
155                 cells[row][col] = random.nextDouble() < density;
156             }
157         }
158     }
159 
160 }
```

Листинг 11: CellularAutomaton.java

## Класс Main

```
1  package ru.spbstu.khramov;
2 
3  import static java.lang.IO.*;
4 
5  public class Main {
6      private Main() {
7      }
8 
9      static void main() {
10         println("Двумерный клеточный автомат");
11         println();
12 
13         int size = readInt("Ширина поля (3...100)", 3, 100, 20);
14         int iterations = readInt("Количество итераций (1...100000)", 1,
               100_000, 10);
15 
16         CellularAutomaton automaton = new CellularAutomaton(43, 11, 2006,
               11, 2, size);
17         initField(automaton);
18 
```

```
19          println();
20          int printEvery = readInt("Печатать каждое k-е поколение, k", 1,
                100_000, 1);
21          boolean stopOnCycle = readInt("Остановиться, когда найден цикл? (1
                - да, 0 - нет)", 0, 1, 1) == 1;
22
23          println();
24          printGeneration(automaton);
25          for (int i = 1; i <= iterations; i++) {
26              boolean hadCycle = automaton.isCycleFound();
27              automaton.step();
28              boolean cycleJustFound = !hadCycle && automaton.isCycleFound()
                    ;
29
30              if (i % printEvery == 0 || i == iterations || cycleJustFound)
                    {
31                  printGeneration(automaton);
32              }
33              if (cycleJustFound && stopOnCycle) {
34                  break;
35              }
36          }
37
38          printSummary(automaton);
39      }
40
41      static void printGeneration(CellularAutomaton automaton) {
42          String header = "Поколение " + automaton.getGeneration() + " | жив
                ых: " + automaton.population();
43          if (automaton.isCycleFound()) {
44              header += " | цикл: предпериод = " + automaton.
                    getTransientLength() + ", период = " + automaton.getPeriod
                    ();
45          }
46          println(header);
47          printField(automaton);
48      }
49
50      static void printSummary(CellularAutomaton automaton) {
51          int size = automaton.getSize();
52          println("Итог");
53          println("Выполнено итераций: " + automaton.getGeneration());
54          println("Живых клеток: " + automaton.population() + " из " + size
                * size);
55          if (automaton.isCycleFound()) {
56              int t = automaton.getTransientLength();
57              int p = automaton.getPeriod();
58              println("Цикл найден на поколении " + (t + p));
```

```
59              println("Предпериод: " + t);
60              println("Период: " + p + (p == 1 ? " (неподвижная точка)" : ""
                    ));
61          } else {
62              println("Цикл за " + automaton.getGeneration() + " итераций не
                    обнаружен.");
63          }
64      }
65
66      static void initField(CellularAutomaton automaton) {
67          println();
68          println("Начальные условия:");
69          println("  1 - случайно");
70          println("  2 - вручную");
71          println("  3 - одна клетка в центре");
72          println("  4 - пустое поле");
73          int mode = readInt("Выбор", 1, 4, 1);
74
75          switch (mode) {
76              case 1 -> {
77                  int percent = readInt("Плотность живых клеток, % (0...100)
                        ", 0, 100, 50);
78                  long seed = readSeed();
79                  automaton.randomize(percent / 100.0, seed);
80              }
81              case 2 -> readManual(automaton);
82              case 3 -> {
83                  int center = automaton.getSize() / 2;
84                  automaton.setCell(center, center, true);
85              }
86              default -> automaton.clear();
87          }
88      }
89
90      static long readSeed() {
91          while (true) {
92              String s = readLine("Seed (пусто = случайный): ");
93              if (s.isEmpty()) {
94                  long seed = System.nanoTime();
95                  println("Используется seed = " + seed);
96                  return seed;
97              }
98              try {
99                  return Long.parseLong(s);
100             } catch (NumberFormatException e) {
101                 println("  Введите целое число или оставьте пустым.");
102             }
103         }
```

```
104     }
105
106     static void readManual(CellularAutomaton automaton) {
107         int size = automaton.getSize();
108         println("Вводите строки поля сверху вниз (до " + size + " строк).");
109         println("Живая клетка: + или 1, мёртвая: . или 0. Пустая строка - закончить ввод.");
110         automaton.clear();
111         for (int row = 0; row < size; row++) {
112             String line = readln(String.format("%3d> ", row));
113             if (line == null || line.isBlank()) {
114                 break;
115             }
116             for (int col = 0; col < Math.min(line.length(), size); col++) {
117                 char ch = line.charAt(col);
118                 automaton.setCell(row, col, ch == '+' || ch == '1');
119             }
120         }
121     }
122
123     static int readInt(String prompt, int min, int max, int def) {
124         while (true) {
125             String s = readLine(prompt + " [" + def + "]: ");
126             if (s.isEmpty()) {
127                 return def;
128             }
129             try {
130                 int value = Integer.parseInt(s);
131                 if (value >= min && value <= max) {
132                     return value;
133                 }
134             } catch (NumberFormatException ignored) {
135             }
136             println("  Введите целое число от " + min + " до " + max + ".");
137         }
138     }
139
140     static String readLine(String prompt) {
141         String s = readln(prompt);
142         return s == null ? "" : s.trim();
143     }
144
145     static void printField(CellularAutomaton automaton) {
146         for (int row = 0; row < automaton.getSize(); row++) {
147             for (int col = 0; col < automaton.getSize(); col++) {
```

```
148                 print(automaton.getCell(row, col) ? "+ " : "  ");
149             }
150             println();
151         }
152         println();
153     }
154 
155 }
```

Листинг 12: Main.java